1. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Kruskal's algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 20
int n;                                  // No. of vertices of G
int cost[MAX][MAX];                     // Cost matrix
int parent[9];
void ReadMatrix() {
   int i, j;
   printf("Enter the no. of vertices\n");
   scanf("%d", &n);
   printf("Enter the cost adjacency matrix\n");
   for (i = 1; i <= n; i++) {
      for (j = 1; j <= n; j++) {
         scanf("%d", &cost[i][j]);
         if (cost[i][j] == 0)
            cost[i][j] = 999;
      }
   }
}
int find(int i) {
   while (parent[i] > 0)
      i = parent[i];
   return i;
}
void uni(int i, int j)
{
   parent[j] = i;
}
void Kruskals()
 {
   int a = 0, b = 0, u = 0, v = 0, i, j, ne = 1, min, mincost = 0;
   printf("The edges of Minimum Cost Spanning Tree are\n");
```

```
   while (ne < n)
{
      for (i = 1, min = 999; i <= n; i++)
{
         for (j = 1; j <= n; j++)
{
            if (cost[i][j] < min)
 {
               min = cost[i][j];
               a = u = i;
               b = v = j;
            }
         }
       }
      u = find(u);
      v = find(v);
      if (u != v)
{
         uni(u, v);
         printf(“ Edge %d:  (%d,%d), Cost = %d\n", ne++, a, b, min);
         mincost += min;
      }
      cost[a][b] = cost[b][a] = 999;
   }
   printf("\nMinimum cost: %d\n", mincost);
}
int main() {
   printf("*****KRUSKAL'S ALGORITHM*****\n");
   ReadMatrix();
   Kruskals();
   return 0;
}
```

Output:

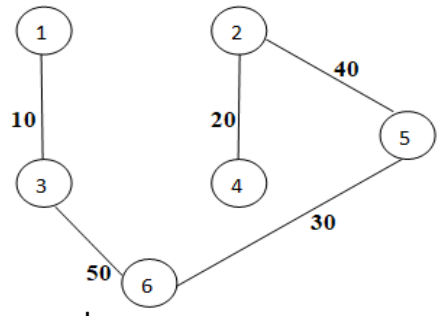
cc 2.c

./a.out

Enter the number of nodes: 6

Enter the adjacency matrix:

0	60	10	999	999	999
60	0	999	20	40	70
10	999	0	999	999	50
999	20	999	0	999	80
999	40	999	999	0	30
999	70	50	80	30	0



The edges of Minimum Cost Spanning Tree are

Edge 1: (1,3) Cost = 10

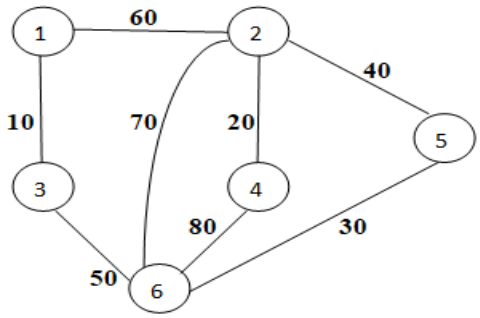
Spanning Tree:

Edge 2: (2,4) Cost = 20

Edge 3: (5,6) Cost = 30

Edge 4: (2,5) Cost = 40

Edge 5: (3,6) Cost = 50



Minimun cost=150

2. Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm.

```
#include<stdio.h>
int u,v,n,i,j,ne=1;
int visited[10]={0},min,mincost=0,cost[10][10];
void main()
{
	printf("\n Enter the number of nodes:\n");
	scanf("%d",&n);
	printf("\n Enter the adjacency matrix:\n");
	for(i=1;i<=n;i++)
	for(j=1;j<=n;j++)
	{
		scanf("%d",&cost[i][j]);
		if(cost[i][j]==0)
		cost[i][j]=999;
	}
	visited[1]=1;
	printf("\n");
	printf("\nThe edges of Minimum Cost Spanning Tree are\n\n");
	while(ne<n)
	{
		for(i=1,min=999;i<=n;i++)
		for(j=1;j<=n;j++)
		if(cost[i][j]<min)
		if(visited[i]!=0)
		{
			min=cost[i][j];
			u=i;
			v=j;
		}
	if(visited[u]==0 || visited[v]==0)
	{
	printf("\n Edge %d:(%d %d) cost:%d",ne++,u,v,min);
```

```
        mincost+=min;
        visited[v]=1;
        }
        cost[u][v]=cost[v][u]=999;
        }
        printf("\n Minimun cost=%d",mincost);
}
```

Output:

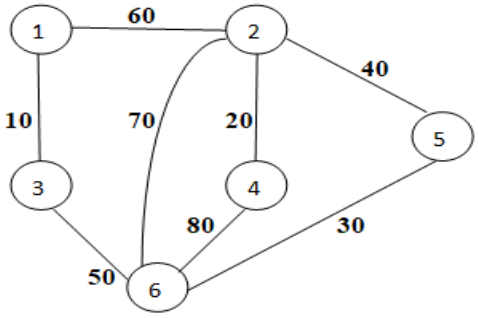
cc 2.c

./a.out

Enter the number of nodes: 6

Enter the adjacency matrix:

0	60	10	999	999	999
60	0	999	20	40	70
10	999	0	999	999	50
999	20	999	0	999	80
999	40	999	999	0	30
999	70	50	80	30	0



The edges of Minimum Cost Spanning Tree are

Edge 1:(1 3) cost:10

Edge 2:(3 6) cost:50

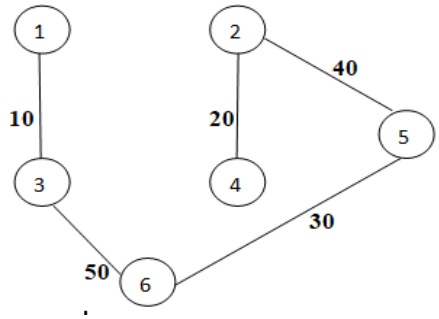
Edge 3:(6 5) cost:30

Edge 4:(5 2) cost:40

Edge 5:(2 4) cost:20

Minimun cost=150

Spanning tree:



3a. Design and implement C/C++ Program to solve All-Pairs Shortest Paths problem using Floyd's algorithm.

```
#include <stdio.h>
#define MAX 20                          // max. size of cost matrix
int a[MAX][MAX];                         // cost matrix
int n, i, j;                            // actual matrix size
void Floyds()
{
   for (int k = 1; k <= n; k++)
      for (int i = 1; i <= n; i++)
         for (int j = 1; j <= n; j++)
            if ((a[i][k] + a[k][j]) < a[i][j])
               a[i][j] = a[i][k] + a[k][j];
}
int main()
{
         printf("*****FLOYD'S ALGORITHM*****\n");
         printf("Enter the number of vertices\n");
         scanf("%d",&n);
         printf("Enter the cost matrix of the graph:\n");
         for(i=1;i<=n;i++)
         for(j=1;j<=n;j++)
         scanf("%d",&a[i][j]);
         Floyds();
         printf("All Pair Shortest path matrix\n");
         for(i=1;i<=n;i++)
         {
                  for(j=1;j<=n;j++)
                  printf("%d ",a[i][j]);
                  printf("\n");
         }
}
```

Output:

```
cc 3a.c
./a.out
```

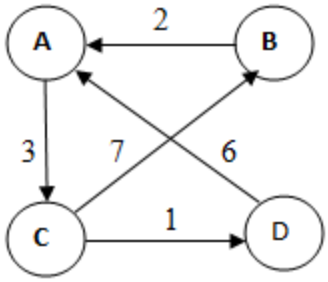
*******FLOYD'S ALGORITHM*******

Enter the number of vertices

4

Enter the cost matrix of the graph:

0	999	3	999
2	0	999	999
999	7	0	1
6	999	999	0



All Pair Shortest path matrix

0	10	3	4
2	0	5	6
7	7	0	16
6	16	9	0

3b. Design and implement C/C++ Program to find the transitive closure using Warshal's algorithm.

```
#include <stdio.h>
#define MAX 20                          // max. size of matrix
int a[MAX][MAX];                        // adjacency matrix
int n, i, j;                            // actual matrix size
void Warshalls() {
   for (int k = 1; k <= n; k++)
      for (int i = 1; i <= n; i++)
         for (int j = 1; j <= n; j++)
            if (a[i][j] == 1 || (a[i][k] == 1 && a[k][j] == 1))
               a[i][j] = 1;
}
int main() {
   printf("*****WARSHALL'S ALGORITHM*****\n");
   printf("Enter the number of vertices\n");
   scanf("%d",&n);
   printf("Enter the adjacency matrix (1 if there is edge and 0 if there is no direct edge)\n");
   for(i=1;i<=n;i++)
   for(j=1;j<=n;j++)
   scanf("%d",&a[i][j]);
   Warshalls();
   printf("The Transitive Closure Matrix is\n");
   for(i=1;i<=n;i++)
   {
   for(j=1;j<=n;j++)
   printf("%d ",a[i][j]);
   printf("\n");
   }
}
```

Output:

cc 3b.c

./a.out

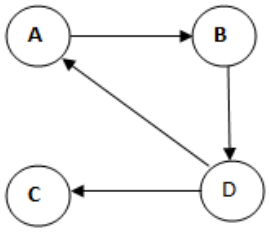
*******WARSHALL'S ALGORITHM*******

Enter the number of vertices

4

Enter the adjacency matrix (1 if there is edge and 0 if there is no direct edge)

0	1	0	0
0	0	0	1
0	0	0	0
1	0	1	0



The Transitive Closure Matrix is

1	1	1	1
1	1	1	1
0	0	0	0
1	1	1	1

4. Design and implement C/C++ Program to find shortest paths from a given vertex in a weighted connected graph to other vertices using Dijkstra's algorithm.

```
#include<stdio.h>
#define INF 999
void dijkstra(int c[10][10],int n,int s,int d[10])
{
	int v[10],min,u,i,j;
	for(i=1;i<=n;i++) {
		d[i]=c[s][i];
		v[i]=0;
	}
	v[s]=1;
	for(i=1;i<=n;i++) {
		min=INF;
		for(j=1;j<=n;j++)
	if(v[j]==0 && d[j]<min) {
		min=d[j];
		u=j;
	}
		v[u]=1;
		for(j=1;j<=n;j++)
		if(v[j]==0 && (d[u]+c[u][j])<d[j])
		d[j]=d[u]+c[u][j];
	}
}
int main()
{
	int c[10][10],d[10],i,j,s,sum,n;
	printf("\nEnter the number of vertices:\n");
	scanf("%d",&n);
	printf("\nEnter the cost adjacency matrix:\n");
	for(i=1;i<=n;i++)
	for(j=1;j<=n;j++)
	scanf("%d",&c[i][j]);
```

```
        printf("\nEnter the source vertex:\n");
        scanf("%d",&s);
        dijkstra(c,n,s,d);
        for(i=1;i<=n;i++)
        if(i!=s)
        printf("\nShortest distance from the path %d --> %d is %d",s,i,d[i]);
         return 0;
}
```

Output:

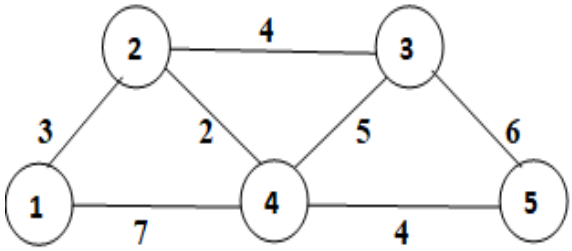
cc 4.c
./a.out

Enter the number of vertices:

5

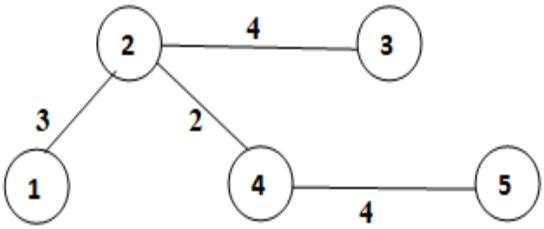
Enter the cost adjacency matrix:

0	**3**	**999**	**7**	**999**
3	**0**	**4**	**2**	**999**
999	**4**	**0**	**5**	**6**
7	**2**	**5**	**0**	**4**
999	**999**	**6**	**4**	**0**



Enter the source vertex:

1



Shortest distance from the path 1 --> 2 is 3
Shortest distance from the path 1 --> 3 is 7
Shortest distance from the path 1 --> 4 is 5
Shortest distance from the path 1 --> 5 is 9

5. Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

```
#include<stdio.h>
int a[10][10],n,indegre[10];
void find_indegre()
{
	 int j,i,sum;
	for(j=0;j<n;j++) {
		sum=0;
	for(i=0;i<n;i++)
	sum+=a[i][j];
	indegre[j]=sum;
	}
}
void topology()
{
	int i,u,v,t[10],s[10],top=-1,k=0;
	find_indegre();
	for(i=0;i<n;i++)
	{
		if(indegre[i]==0)
		s[++top]=i;
	}
	while(top!=-1)
	{
		u=s[top--];
		t[k++]=u;
	for(v=0;v<n;v++)
	{
	if(a[u][v]==1)
	{
		indegre[v]--;
		if(indegre[v]==0)
		s[++top]=v;
```

```
        }
        }
        }
        printf("The topological Sequence is:\n");
        for(i=0;i<n;i++)
        printf("%d ",t[i]);
}
void main()
{
        int i,j;
        printf("Enter number of jobs:");
        scanf("%d",&n);
        printf("\nEnter the adjacency matrix:\n");
        for(i=0;i<n;i++)
        {
        for(j=0;j<n;j++)
        scanf("%d",&a[i][j]);
        }
        topology();
}
```

Output:

cc 5.c

./a.out

Enter number of jobs:6

Enter the adjacency matrix:

0 0 1 1 0 0

0 0 0 1 1 0

0 0 0 1 0 1

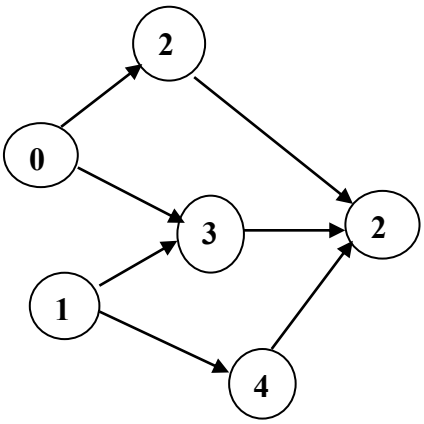
0 0 0 0 0 1

0 0 0 0 0 1

0 0 0 0 0 0

The topological Sequence is:

1 4 0 2 3 5



6. Design and implement C/C++ Program to solve 0/1 Knapsack problem using Dynamic Programming method.

```
#include<stdio.h>
int w[10],p[10],v[10][10],n,i,j,cap,x[10]={0};
int max(int i,int j)
{
	return ((i>j)?i:j);
}
int knap(int i,int j)
{
	int value;
	if(v[i][j]<0)
	{
		if(j<w[i])
		value=knap(i-1,j);
		else
		value=max(knap(i-1,j),p[i]+knap(i-1,j-w[i]));
		v[i][j]=value;
	}
	return(v[i][j]);
}
void main()
{
	int profit,count=0;
	printf("\nEnter the number of elements\n");
	scanf("%d",&n);
	printf("Enter the profit and weights of the elements\n");
	for(i=1;i<=n;i++)
	{
		printf("For item no %d\n",i);
		scanf("%d\t%d",&p[i],&w[i]);
	}
	printf("\nEnter the capacity \n");
	scanf("%d",&cap);
```

```
        for(i=0;i<=n;i++)
        for(j=0;j<=cap;j++)
        if((i==0)||(j==0))
                v[i][j]=0;
                else
                v[i][j]=-1;
                profit=knap(n,cap);
                i=n;
                j=cap;
        while(j!=0&&i!=0)
        {
                if(v[i][j]!=v[i-1][j])
                {
                        x[i]=1;
                        j=j-w[i];
                        i--;
                }
                else
                i--;
        }
        printf("\nItems included are\n");
        printf("Sl.no\t weight\t profit\n");
        for(i=1;i<=n;i++)
        if(x[i])
        printf("%d\t\t %d\t\t %d\n",++count,w[i],p[i]);
        printf("Total profit = %d\n",profit);
}
```

Output:

```
        cc 6.c
        ./a.out
Enter the number of elements
4
Enter the profit and weights of the elements
For item no 1
```

12 2

For item no 2

10 1

For item no 3

20 3

For item no 4

15 2

Enter the capacity

5

Items included are

Sl.no	weight	profit
1	2	12
2	1	10
3	2	15

Total profit = 37

7. Design and implement C/C++ Program to solve discrete Knapsack and continuous Knapsack problems using greedy approximation method.

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
   float w;
   float p;
   float r;
} KObject;
int n;              // no. of objects
float M;       // capacity of Knapsack

void ReadObjects(KObject obj[])
{
   KObject temp;
   printf("Enter the max capacity of knapsack: ");
   scanf("%f", &M);
   printf("Enter Weights: ");
   for (int i = 0; i < n; i++)
      scanf("%f", &obj[i].w);
   printf("Enter Profits: ");
   for (int i = 0; i < n; i++)
      scanf("%f", &obj[i].p);
   for (int i = 0; i < n; i++)
      obj[i].r = obj[i].p / obj[i].w;
   for (int i = 0; i < n - 1; i++)
      for (int j = 0; j < n - 1 - i; j++)
         if (obj[j].r < obj[j + 1].r) {
            temp = obj[j];
            obj[j] = obj[j + 1];
            obj[j + 1] = temp;
         }
}
void Knapsack(KObject kobj[])
```

```
{
   float x[20];
   float totalprofit;
   int i;
   float rc;                // U place holder for M
   rc = M;
   totalprofit = 0;
   for (i = 0; i < n; i++)
      x[i] = 0;
   for (i = 0; i < n; i++) {
      if (kobj[i].w >= rc)
         break;
      else
      {
         x[i] = 1;
         totalprofit = totalprofit + kobj[i].p;
         rc = rc - kobj[i].w;
      }
   }
   if (i < n)
      x[i] = rc / kobj[i].w;
   totalprofit = totalprofit + (x[i] * kobj[i].p);
   printf("The Solution is: \n");
   printf("weight\tprofit\tFraction\t\n");
   for (i = 0; i < n; i++)
      printf("%.2f\t%.2f\t%.2f\n", kobj[i].w, kobj[i].p, x[i]);
   printf("\nTotal profit is = %.2f\n", totalprofit);
}

int main()
{
   printf("*****KNAPSACK USING GREEDY METHOD*****\n");
   printf("Enter number of objects: ");
   scanf("%d", &n);
   KObject obj[n];
```

```
    ReadObjects(obj);
    Knapsack(obj);
    return 0;
}
```

Output:

```
cc 7.c
./a.out
```

```
*****KNAPSACK USING GREEDY METHOD*****
Enter number of objects: 3
Enter the max capacity of knapsack: 40
Enter Weights: 20 25 10
Enter Profits: 30 40 35
The Solution is:
Weight   Profit    Fraction
 10.00    35.00      1.00
 25.00    40.00      1.00
 20.00    30.00      0.25

Total profit is = 82.50
```

8. Design and implement C/C++ Program to find a subset of a given set S = {sl , s2,.....,sn} of n positive integers whose sum is equal to a given positive integer d.

```
#include<stdio.h>
#define MAX 10
int s[MAX],x[MAX],d;
void sumofsub(int p,int k,int r) {
        int i;
        x[k]=1;
        if((p+s[k])==d) {
                for(i=1;i<=k;i++)
                if(x[i]==1)
                printf("%d ",s[i]);
                printf("\n");
}
else
        if(p+s[k]+s[k+1]<=d)
        sumofsub(p+s[k],k+1,r-s[k]);
        if((p+r-s[k]>=d) && (p+s[k+1]<=d))
        {
                x[k]=0;
                sumofsub(p,k+1,r-s[k]);
        }
}
int main() {
        int i,n,sum=0;
        printf("Enter the n value:");
        scanf("%d",&n);
        printf("Enter the set in increasing order:");
        for(i=1;i<=n;i++)
        scanf("%d",&s[i]);
        printf("Enter the max subset value:");
        scanf("%d",&d);
        for(i=1;i<=n;i++)
        sum=sum+s[i];
```

```
        if(sum<d || s[1]>d)
        printf("No subset possible");
        else
        sumofsub(0,1,sum);
        return 0;
}
```

Output:

cc 8.c

./a.out

1. **Enter the n value: 5**
   Enter the set in increasing order: 1 2 5 6 8
   Enter the max subset value: 9
   1 2 6
   1 8

2. **Enter the n value: 5**
   Enter the set in increasing order: 1 3 4 5 6
   Enter the max subset value: 20
   No subset possible

3. **Enter the n value: 9**
   Enter the set in increasing order: 1 2 3 4 5 6 7 8 9
   Enter the max subset value: 9
   1 2 6
   1 3 5
   1 8
   2 3 4
   2 7
   3 6
   4 5
   9

9. Design and implement C/C++ Program to sort a given set of n integer elements using Selection Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>
void selsort(int a[],int n)
{
	int i,j,small,pos,temp;
	for(i=0;i<n-1;i++)
	{
		small=a[i];
		pos=i;
	for(j=i+1;j<n;j++)
	{
	if(a[j]<small)
	{
		small=a[j];
		pos=j;
	}
	}
		temp=a[i];
		a[i]=small;
		a[pos]=temp;
	}
}
void main()
{
	int a[10],i,n;
	clock_t start,end;
	printf("\nEnter the n value:");
	scanf("%d",&n);
	for(i=0;i<n;i++)
	 a[i] = 5000 + rand() % 999;		// generate random numbers
```

```
        printf("Input Array:\n");
        for (int i = 0; i < n; i++)
        printf("%d\t ", a[i]);
        printf("\n");
        start=clock();
        selsort(a,n);
        end = clock();
        double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC * 1000; // in milliseconds
        printf("\nSorted array is:\n");
        for(i=0;i<n;i++)
        printf("%d\t ",a[i]);
        printf("\nTime taken is:%f",time_taken);
}
```

Output:

```
cc 9.c
./a.out

Enter the n value:5
Input Array:
5478    5664    5153    5268    5500
Sorted array is:
5153    5268    5478    5500    5664
Time taken is:0.002000
```

10. Design and implement C/C++ Program to sort a given set of n integer elements using Quick Sort method and compute its time complexity. Run the program for varied values of n> 5000 and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 10005
int a[MAX];
void QuickSortAlgorithm(int p, int r) {
   int i, j, temp, pivot;
   if (p < r) {
      i = p;
      j = r + 1;
      pivot = a[p];                  // mark first element as pivot
      while (1) {
         i++;
         while (a[i] < pivot && i < r)
            i++;
         j--;
         while (a[j] > pivot)
            j--;
         if (i < j) {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
         } else
            break;                   // partition is over
      }
      a[p] = a[j];
      a[j] = pivot;
      QuickSortAlgorithm(p, j - 1);
      QuickSortAlgorithm(j + 1, r);
   }
}
```

```
int main() {
   int n;
   printf("***** QUICK SORT***** \n");
   printf("Enter Max array size: ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++)
    a[i] = 5000 + rand() % 999;                    // generate random numbers
   printf("Input Array:\n");
   for (int i = 0; i < n; i++)
    printf("%d\t ", a[i]);
   printf("\n");
   clock_t start = clock();
   QuickSortAlgorithm(0, n - 1);
   clock_t end = clock();
   double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC * 1000; // in milliseconds
   printf("Sorted Array:\n");
   for (int i = 0; i < n; i++)
      printf("%d\t ", a[i]);
   printf("\nTime Complexity in ms for n=%d is: %.6f\n", n, time_taken);
   return 0;
}
```

Output:

```
cc 10.c
./a.out

***** QUICK SORT*****
Enter Max array size: 5
Input Array:
5478 5664 5153 5268 5500
Sorted Array:
5153 5268 5478 5500 5664
Time Complexity in ms for n=5 is: 0.001000
```

11. Design and implement C/C++ Program to sort a given set of n integer elements using Merge Sort method and compute its time complexity. Run the program for varied values of n> 5000, and record the time taken to sort. Plot a graph of the time taken versus n. The elements can be read from a file or can be generated using the random number generator.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define MAX 10005
int a[MAX];
void MergeSort(int low, int high);
void Merge(int low, int mid, int high);
int main() {
 int n;
 printf("***** MERGE SORT***** \n");
 printf("Enter Max array size: ");
 scanf("%d", &n);
 for (int i = 0; i < n; i++)
    a[i] = 5000 + rand() % 999;                    // generate random numbers
    printf("Input Array:\n");
   for (int i = 0; i < n; i++)
    printf("%d\t ", a[i]);
    printf("\n");
    clock_t start = clock();
    MergeSort(0, n - 1);
    clock_t end = clock();
  double elapsedTime = ((double)(end - start)) / CLOCKS_PER_SEC * 1000;
 printf("Sorted Array (Merge Sort):\n");
 for (int i = 0; i < n; i++)
  printf("%d\t ", a[i]);
  printf("\n Time Complexity (ms) for n = %d is : %f\n", n, elapsedTime);
  printf("\n");
  return 0;
}

void MergeSort(int low, int high) {
```

```
    int mid;
    if (low < high) {
        mid = (low + high) / 2;
        MergeSort(low, mid);
        MergeSort(mid + 1, high);
        Merge(low, mid, high);
    }
}
void Merge(int low, int mid, int high) {
    int b[MAX];
    int i, h, j, k;
    h = i = low;
    j = mid + 1;
    while ((h <= mid) && (j <= high)) {
        if (a[h] < a[j]) {
            b[i++] = a[h++];
        } else {
            b[i++] = a[j++];
        }
    }
    if (h > mid) {
        for (k = j; k <= high; k++) {
            b[i++] = a[k];
        }
    } else {
        for (k = h; k <= mid; k++) {
            b[i++] = a[k];
        }
    }
    for (k = low; k <= high; k++) {
        a[k] = b[k];
    }
}
```

Output:

```
cc 11.c
./a.out

***** MERGE SORT*****
Enter Max array size: 5
Input Array:
5478 5664 5153 5268 5500
Sorted Array (Merge Sort):
5153    5268    5478    5500    5664
 Time Complexity (ms) for n = 5 is : 0.006000
```

12. Design and implement C/C++ Program for N Queen's problem using Backtracking.

```
#include<stdio.h>
#include<math.h>
#include<stdlib.h>
int a[30],count=0;
int place(int pos)
{
	int i;
	for(i=1;i<pos;i++)
	{
		if((a[i]==a[pos])||((abs(a[i]-a[pos])==abs(i-pos))))
		return 0;
	}
	return 1;
}
void print_sol(int n)
{
	int i,j;
	count++;
	printf("\n\nSolution #%d:\n",count);
	for(i=1;i<=n;i++)
	{
		for(j=1;j<=n;j++)
		{
			if(a[i]==j)
			printf("Q\t");
			else
			printf("*\t");
		}
		printf("\n");
	}
}
```

```
void queen(int n)
{
        int k=1;
        a[k]=0;
        while(k!=0)
        {
                a[k]=a[k]+1;
                while((a[k]<=n)&&!place(k))
                a[k]++;
                if(a[k]<=n)
                {
                        if(k==n)
                        print_sol(n);
                        else
                        {
                                k++;
                                a[k]=0;
                        }
                }
                else
                k--;
        }
}
void main()
{
        int i,n;
        printf("Enter the number of Queens\n");
        scanf("%d",&n);
        queen(n);
        printf("\nTotal solutions=%d",count);
}
```

Output:

```
cc 12.c
./a.out

Enter the number of Queens
4

Solution #1:
*   Q   *   *
*   *   *   Q
Q   *   *   *
*   *   Q   *

Solution #2:
*   *   Q   *
Q   *   *   *
*   *   *   Q
*   Q   *   *

Total solutions=2
```